

Code Rush — Web Dev Track

Time limit: 8 hours **Theme:** Real-time crisis ops, maps, and AI

The scenario

A major shipping chokepoint just got flagged as a high-risk red zone after sudden geopolitical instability and naval blockades. Fifteen commercial cargo ships are in transit through the area right now under uncertain conditions, and the global supply chain is already feeling it.

You're building the command system that keeps the fleet visible, controllable, and alive: live ship tracking, role-based control between fleet command and ship captains, AI for distress calls, weather-aware rerouting, and alerts when ships drift into trouble.

It's a lot. Build the core first.

The numbers (fixed for grading)

- Exactly 15 active ships
- Ship updates come in at 1 Hz or faster
- Ship state reaches everyone watching within 500ms, 95% of the time
- Geofence breach alerts fire within 1 second of the boundary cross
- Proximity warnings trigger when two ships get within 2 km
- Adverse weather costs 30% extra fuel
- At least 5 people can watch the same fleet at once without state going out of sync
- Ships move smoothly between updates with no teleporting

How ships move

There's no real fleet out there sending data. You're writing the simulator yourself. Each of the 15 ships is just state on your backend that gets advanced over time.

Every ship has:

- A current position (lat/lng)
- A speed and heading
- A destination port
- Fuel remaining
- Cargo
- An operational status (normal, rerouting, distressed, stopped, etc.)

Routing is your job. We don't bake routes into the fleet config. Each ship starts at a position and knows where it's going, but the path between them is something your system has to figure out. The path needs to stay in navigable water (a polygon we provide, so you don't have to deal with real coastline data) and avoid any restricted zones operators have drawn at runtime.

Two things trigger a reroute:

- A new zone is drawn that intersects the ship's current path. The ship's status flips to "rerouting," your routing logic computes a new path that avoids the zone (and any others), and the ship follows it. No operator approval needed because the goal is unambiguous.
- The captain accepts a directive from Command that changes course (reroute to a different port, divert to a waypoint, hold position). The next tick's route computation picks up the change.

Weather isn't a trigger by itself. Ships in bad weather pay the 30% fuel penalty and keep going. If Command wants a ship to avoid a storm, that's a regular reroute directive, and the routing logic will naturally pick a path around the worst of it.

The routing algorithm itself is up to you. A* on a grid, visibility graph, naive "head toward destination and deflect when blocked" — all valid. We're testing the behavior, not the algorithm.

A few edge cases to handle so your simulator doesn't get stuck:

- If no valid path exists (zones fully box in the destination), the ship goes to "stranded" status and an alert fires
- If a zone is drawn around a ship that's already inside it, fire the geofence breach alert and have the ship attempt to reroute out
- If a ship doesn't have enough fuel to reach its destination on the new path, flag "insufficient fuel" status but let it keep going until it actually runs out

Your backend runs a tick at 1 Hz or faster. On each tick:

- Advance every ship along its current heading by (speed × tick duration)
- Apply the 30% extra fuel burn when the ship is in adverse weather
- Update each ship's status based on conditions (out of fuel, inside a zone, arrived, etc.)
- Push the new state out to everyone watching

The operational area is the Strait of Hormuz and surrounding waters — Persian Gulf to the west, Gulf of Oman to the east. We provide `fleet.json` with the bounding box, a simplified navigable water polygon, and the 15 starting ships with their destination ports. Every team grades on the same fleet.

What you're building

The map

- Show all 15 ships on an interactive ocean map
- From the Command interface, operators can draw and edit polygonal restricted zones at runtime. Captains see the zones on their map but can't add or modify them.
- When a ship enters a restricted zone, fire an alert (visual and audible) and keep it active until someone acknowledges or resolves it
- Clicking a ship shows its cargo, fuel, speed, destination, and operational status

Real-time sync

- Use a persistent connection like WebSocket. No polling.
- Everyone connected sees the same fleet state
- Interpolate ship movement between updates so motion looks continuous
- Keep that interpolation physically plausible: respect a max speed, no position jumps

Roles

There are two interfaces:

- **Command** sees the whole fleet, can issue directives to any ship, and is the only role that can draw or edit restricted zones
- **Captain** is scoped to a single ship. Receives directives and responds with either ACCEPT (ship adopts the new course on the next tick) or ESCALATE_DISTRESS (sends a free-form distress message instead of complying, which the AI then processes for severity and impact)

When command sends a directive, the relevant captain responds, and that response reaches everyone connected immediately.

AI / NLP

- Captains can type free-form distress messages
- Your system pulls structured info out of those messages: severity, what's wrong, quantifiable impact like injury counts or damage estimates
- That extracted data feeds into how alerts get prioritized and displayed across the system

Weather and routing

- Pull real weather data from Open-Meteo, Stormglass, or any service with a free tier
- Apply the 30% fuel penalty whenever a ship is moving through adverse weather

- When the routing logic computes a new path (for any reason), factor in current weather so the path avoids the worst of it
- Tell us whether the ship can actually reach its destination on the fuel it has, given the path and conditions

Proximity warnings

- Continuously check pairs of ships
- Warn when any two ships are within 2 km of each other
- Run those warnings through the same alert pipeline as everything else

Playback

- Save state over time however you want: event log, periodic snapshots, ring buffer, your call
- Build a timeline so users can scrub back through fleet positions and key events
- Last hour of history at 30-second resolution is enough
- We don't need full state reconstruction at arbitrary timestamps

Running it

- `docker compose up` should bring the whole system up
- Document any API keys or environment variables in the README

Bonuses

Only worth touching once core is stable. Each is up to +5%, additive, capped at 110% total. These are tiebreakers, not a way to skip the core requirements.

- Multiple route options on reroute. Generate a few candidate paths with different tradeoffs (faster but through bad weather, longer but safer, most fuel-efficient) and let the operator pick.
- Ship-to-ship assistance. A ship in distress can request help from nearby ships in the fleet — fuel transfer, medical aid, escort, cargo offload — and the receiving captain accepts or declines.
- Predictive alerts that fire before things go wrong, not after. "Ship will enter the new zone in 3 minutes." "Ship will run out of fuel 40 km short of port."
- AI fleet advisor. Beyond reacting to distress messages, the AI proactively suggests actions to Command (which ship to reroute, where to consider drawing a zone, who could send aid) with reasoning the operator can accept or reject.

Grading

Criterion	Weight
Core functionality end-to-end, with consistent state across everyone connected	60%
AI / NLP integration depth	20%
Geospatial accuracy and UI/UX	15%
Code quality, structure, and understanding of the tech stack	5%

Ground rules

- The system has to run end-to-end on a laptop. No reliance on cloud infra you can't reproduce during judging.
- Hard-coded data is only acceptable where the spec explicitly allows it, like basemap tiles. Ship data, alerts, and AI outputs all have to be live.
- If the spec didn't say something, document the assumption you made. Documented assumptions will be honored. Undocumented ones will be judged against the strictest reasonable interpretation.